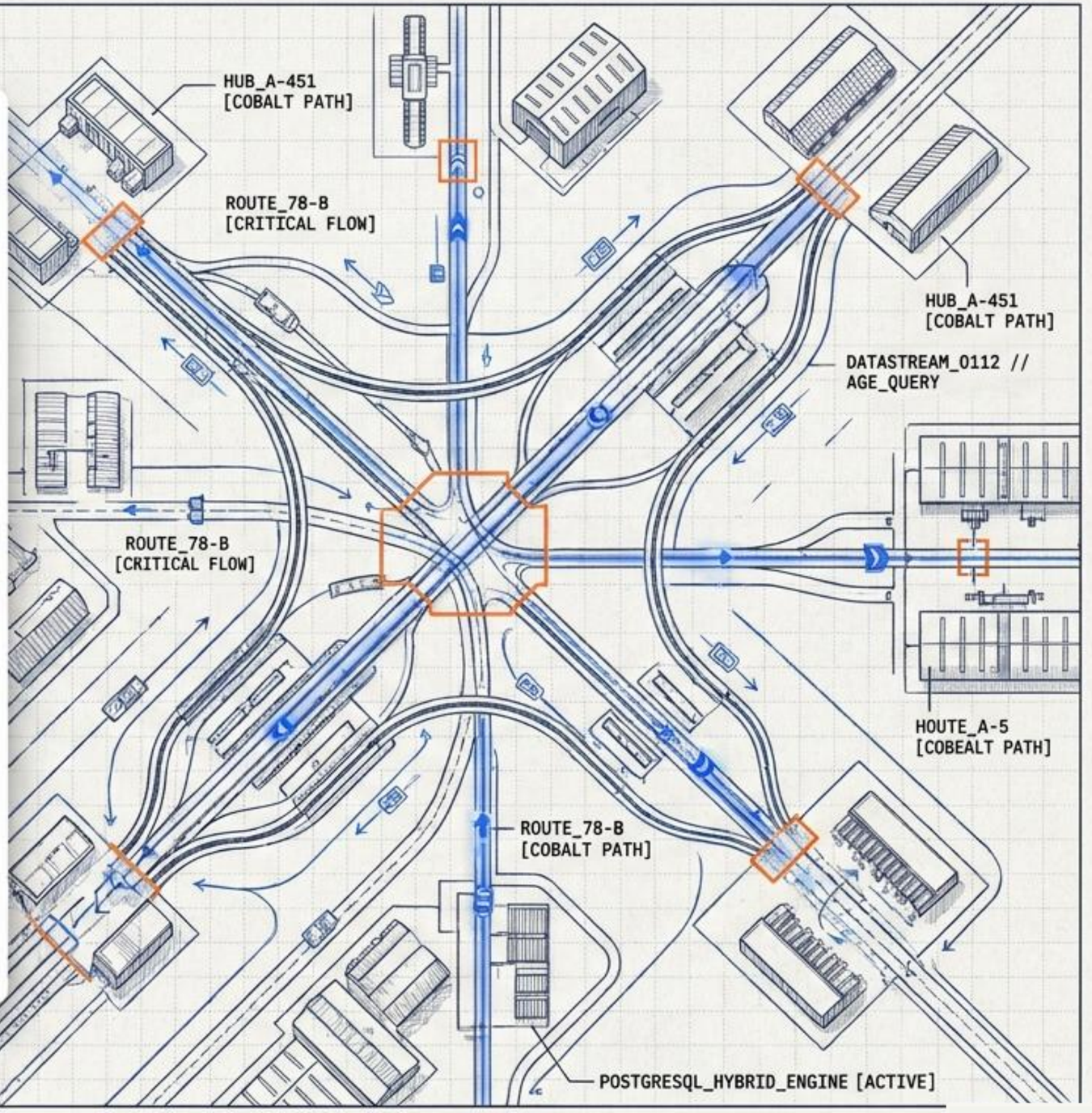
MÓDULO 1 // SEMANA 3: REDES COMPLEXAS

O Paradigma de Grafos na Logística

Modelagem, Traversals e o Motor Híbrido com Apache AGE e PostgreSQL.

Treinamento Avançado em Banco de Dados

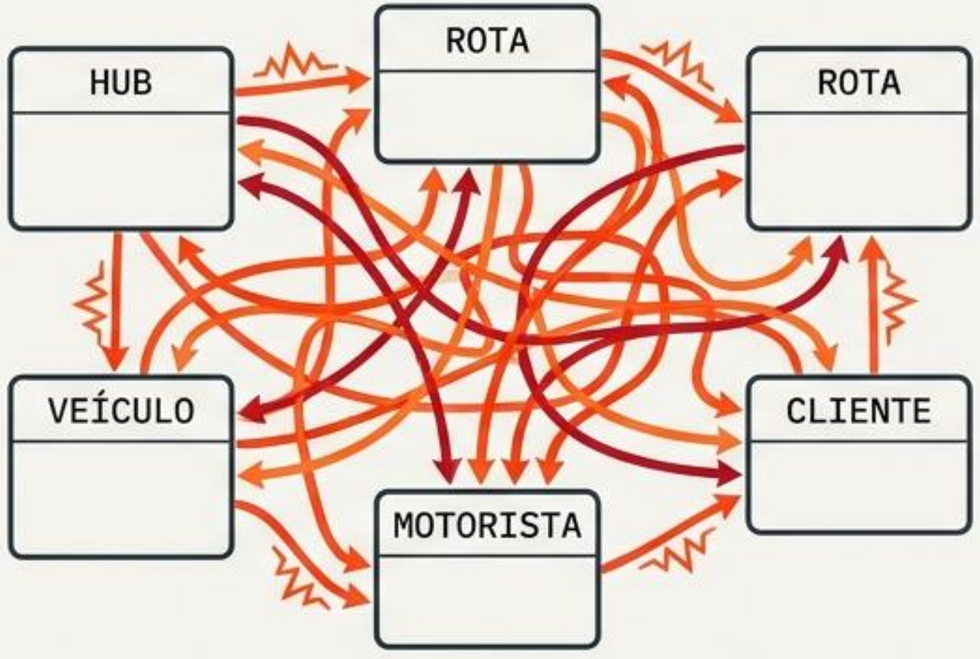


O Limite do Relacional em Redes Profundas

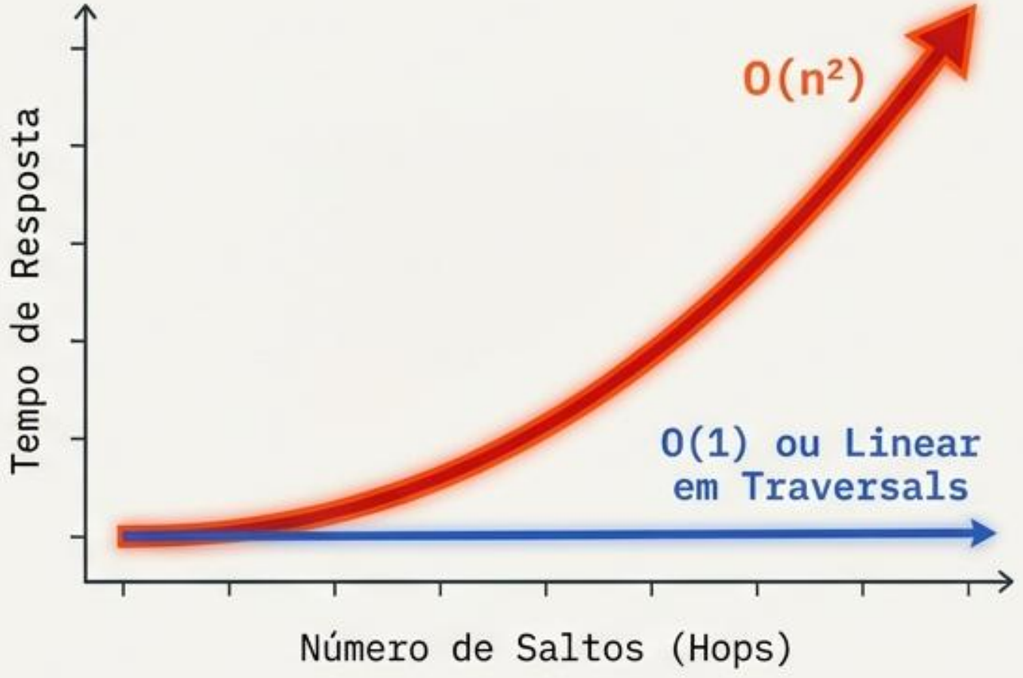
Em SQL, encontrar todas as rotas entre o Hub A e o Cliente B com até 5 hubs intermediários exige múltiplos JOINs. O custo computacional cresce exponencialmente.

Precisamos de um modelo onde os relacionamentos sejam cidadãos de primeira classe, não operações calculadas em tempo de execução.

O Efeito Spaghetti JOIN

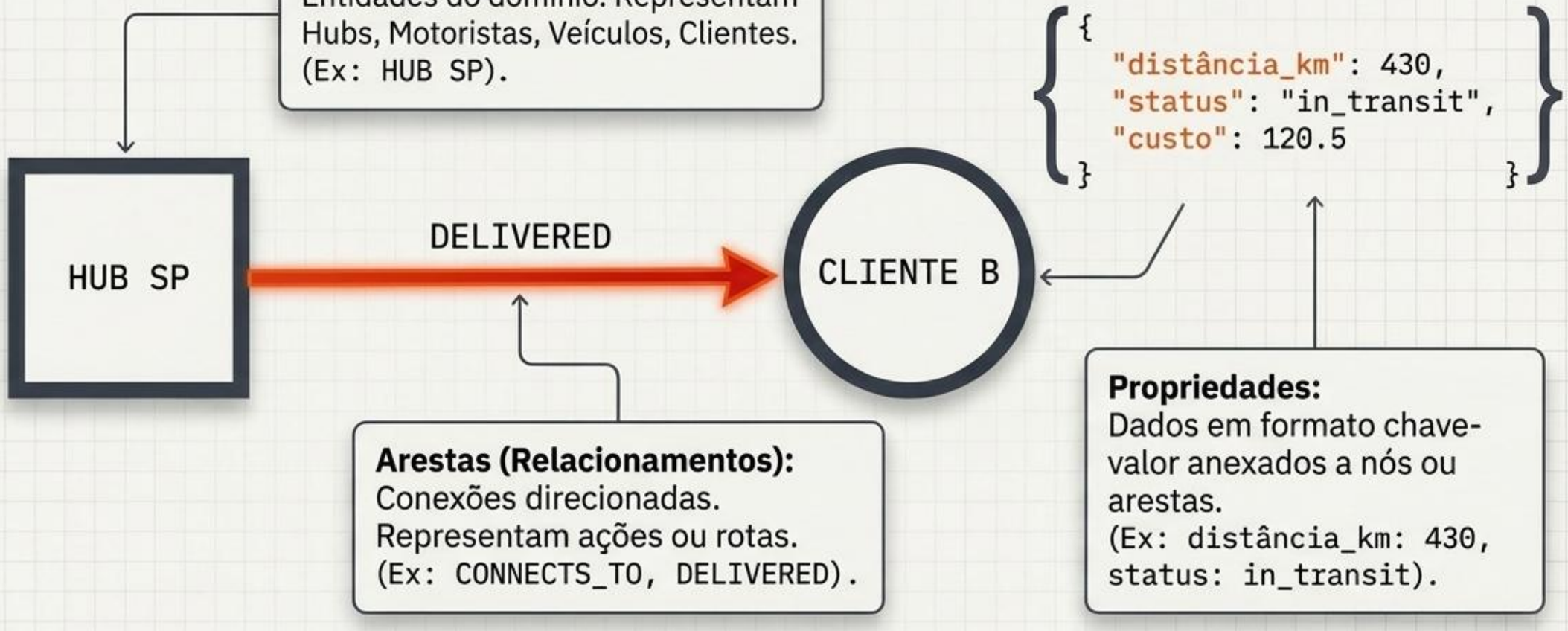


Complexidade de Busca



A Anatomia do Grafo Logístico

Vértices (Nós):
Entidades do domínio. Representam Hubs, Motoristas, Veículos, Clientes.
(Ex: HUB SP).



Arestas (Relacionamentos):
Conexões direcionadas.
Representam ações ou rotas.
(Ex: CONNECTS_TO, DELIVERED).

Propriedades:
Dados em formato chave-valor anexados a nós ou arestas.
(Ex: distância_km: 430, status: in_transit).

Grafos podem ser direcionados (A → B) e ponderados (Arestas possuem pesos como distância ou custo).

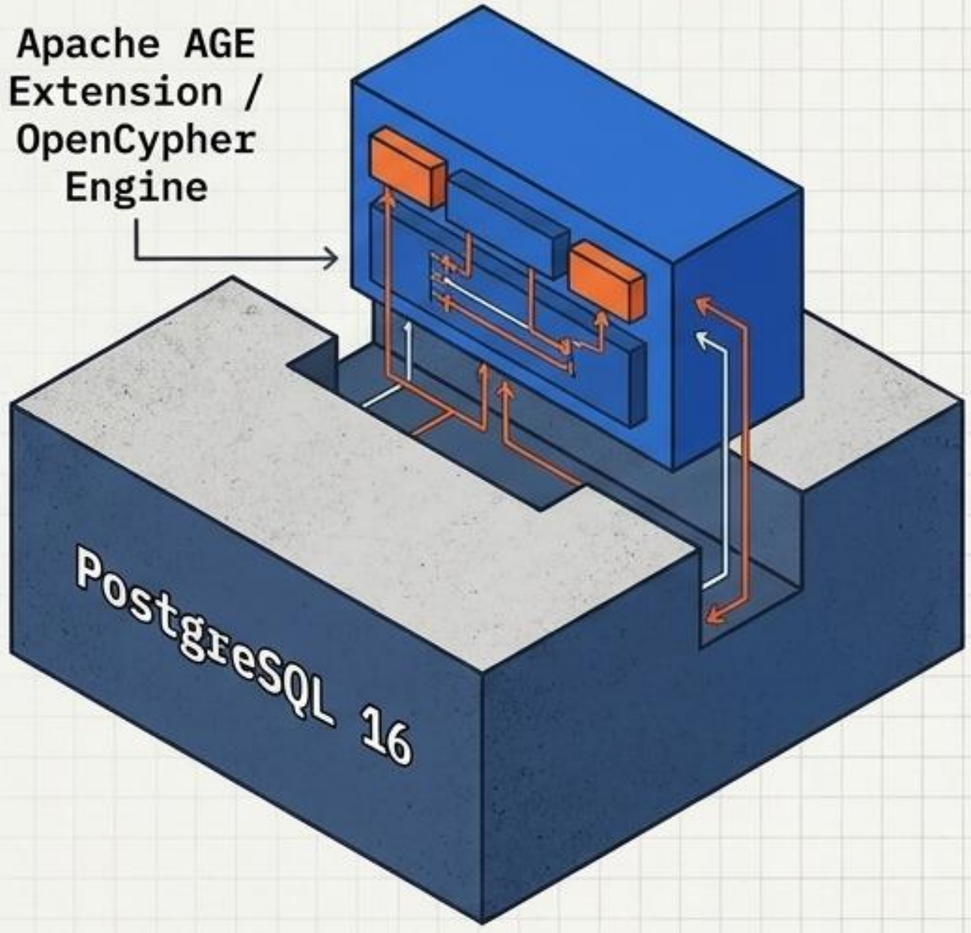
Matriz de Decisão Arquitetural: Quando usar o quê?

Dimensão	Relacional (SQL)	Documento (NoSQL)	Grafos (Cypher)
Estrutura de Dados	Estruturados	Semiestruturados	Redes Complexas
Integridade Transacional	Forte (ACID)	Eventual	Forte (AGE/Neo4j)
Flexibilidade de Schema	Rígida	Alta (Variável)	Alta
Complexidade de Busca (Traversals)	Baixa (JOINs lentos)	Muito Baixa	Excelente
Caso de Uso Ideal	Transações Financeiras	Catálogos Variáveis	Roteamento e Análise de Redes

Apache AGE: O Motor Híbrido (Graph + ACID)

Por que abandonar a estabilidade do PostgreSQL? Com o Apache AGE, você não precisa.

- **Arquitetura Nativa**: Extensão que injeta suporte a grafos diretamente no PostgreSQL.
- **Linguagem Cypher**: Utiliza o padrão openCypher (sintaxe declarativa similar ao Neo4j).
- **Confiabilidade**: Mantém 100% da garantia ACID do Postgres.
- **Aplicações**: Roteamento logístico, detecção de fraudes, sistemas de recomendação.



Ambiente Técnico: Setup e Inicialização

docker-compose.yml

```yaml
version: '3.8'
services:
  postgres-age:
    image: apache/age:latest
    environment:
      POSTGRES_DB: swifttrack_graph
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: secret
    ports:
      - '5433:5432'
```

init_age.sql

```sql
-- Habilitar extensão nativa
CREATE EXTENSION IF NOT EXISTS age;
LOAD 'age';

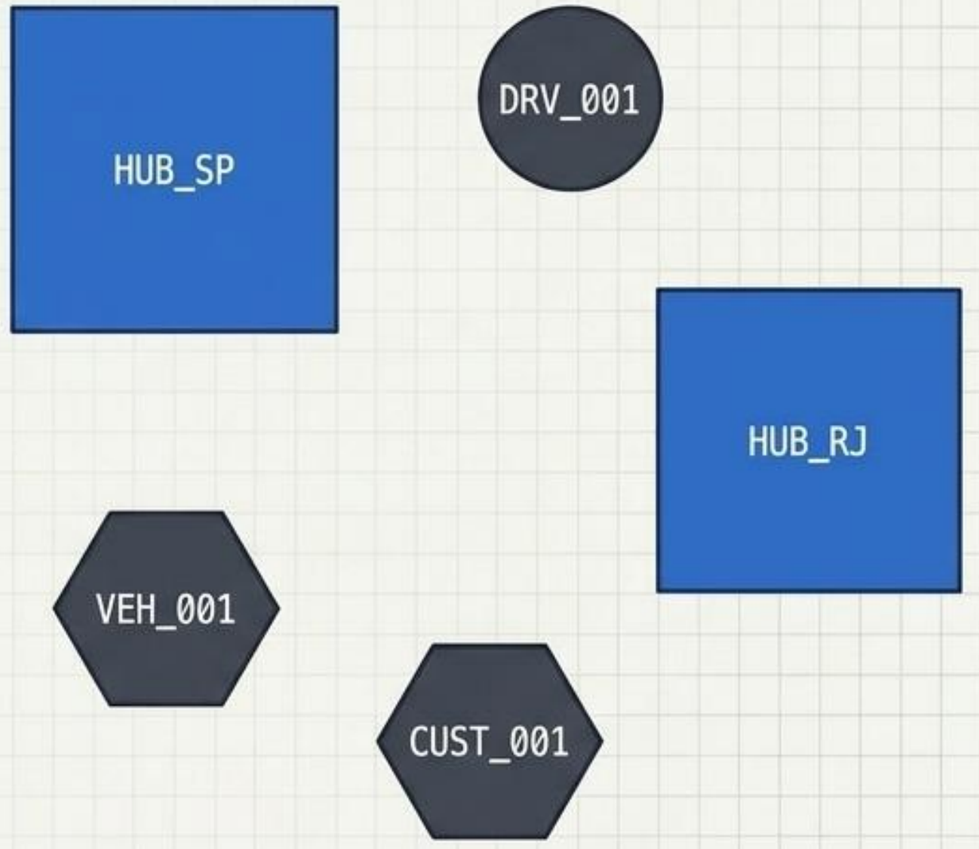

-- Inicializar o ambiente de grafos
SELECT create_graph('logistics_network');
```

Modelagem 01: Instanciando Entidades (Vértices)

1
```
-- [1] Criar Hubs Logísticos
CREATE (:Hub {id: 'HUB_SP', name: 'São Paulo
Central', capacity: 5000});
CREATE (:Hub {id: 'HUB_RJ', name: 'Rio de
Janeiro Central', capacity: 3000});
```

2
```
-- [2] Criar Atores e Recursos
CREATE (:Driver {id: 'DRV_001', name: 'Carlos
Silva', rating: 4.8});
CREATE (:Vehicle {id: 'VEH_001', type: 'Van',
capacity_kg: 1000});
CREATE (:Customer {id: 'CUST_001', name:
'Empresa ABC', type: 'B2B'});
```

A sintaxe (:Label {propriedades}) cria a entidade no espaço dimensional do grafo. Neste momento, os nós existem, mas a rede ainda está inerte.



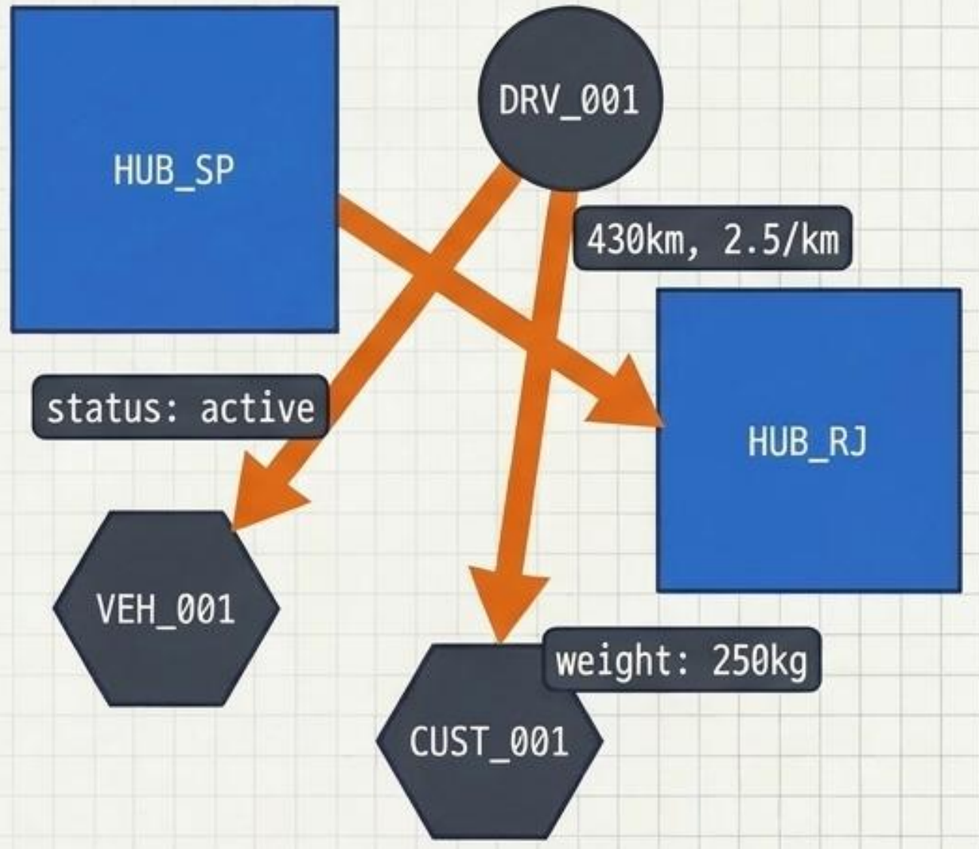
Modelagem 02: Estabelecendo a Rede (Arestas)

```
-- Conectar Hubs (Rotas Ponderadas)
MATCH (a:Hub {id: 'HUB_SP'}), (b:Hub {id: 'HUB_RJ'})
CREATE (a)-[:CONNECTS_TO {distance_km: 430,
cost_per_km: 2.5}]->(b);

-- Associar Motorista ao Veículo
MATCH (d:Driver {id: 'DRV_001'}), (v:Vehicle {id:
'VEH_001'})
CREATE (d)-[:ASSIGNED_TO {status: 'active'}]->(v);

-- Registrar Entrega
MATCH (d:Driver {id: 'DRV_001'}), (c:Customer {id:
'CUST_001'})
CREATE (d)-[:DELIVERED {weight_kg: 250}]->(c);
```

O padrão (nó_origem)-[:RELACIONAMENTO]->(nó_destino) define o fluxo e a topologia. Propriedades nas arestas são essenciais para algoritmos de roteamento.

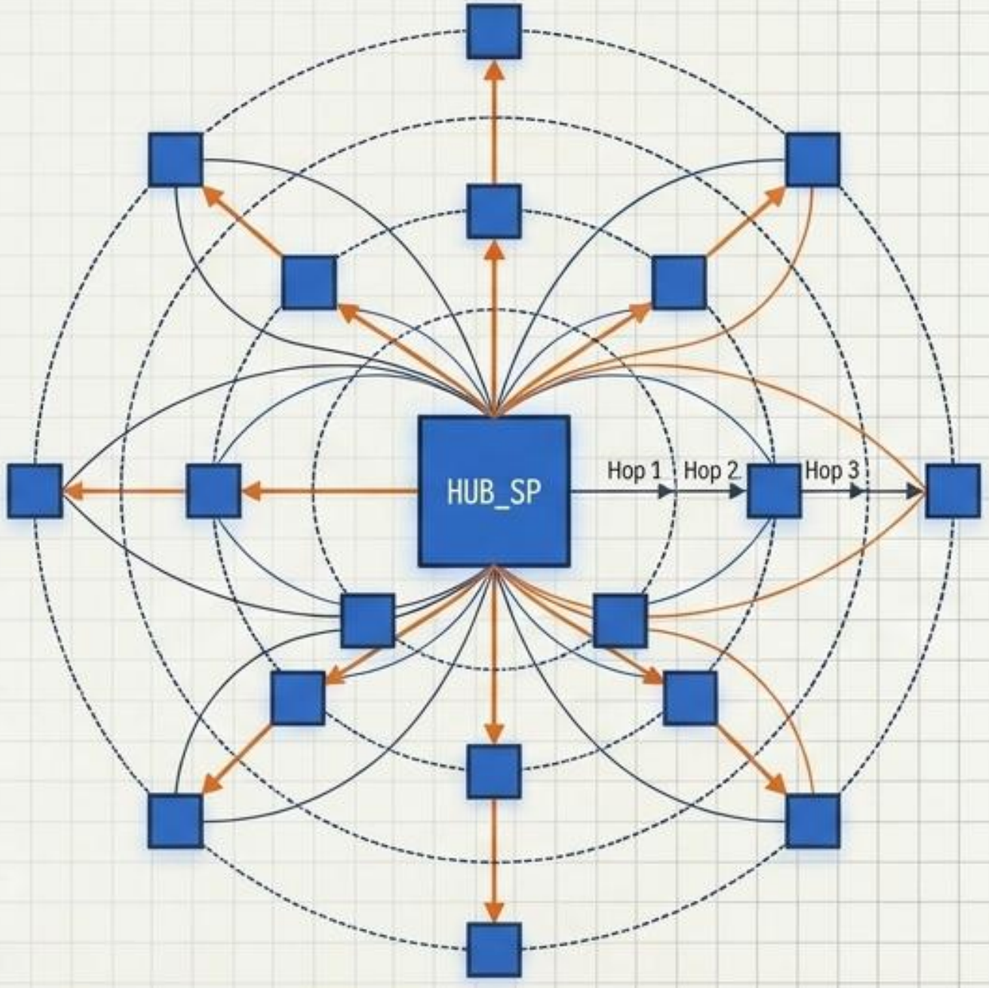


Query 1: O Poder do Traversal Profundo (3-Hops)

Encontrar todos os motoristas que entregaram em um raio de 3 saltos de São Paulo.

```
MATCH (hub:Hub {id: 'HUB_SP'})-
[:CONNECTS_TO*1..3]->(connectedHub)
MATCH (driver:Driver)-[:DELIVERED]->
(customer:Customer)
WHERE customer.city = connectedHub.city
RETURN DISTINCT driver.name, count(customer)
as total_deliveries
ORDER BY total_deliveries DESC;
```

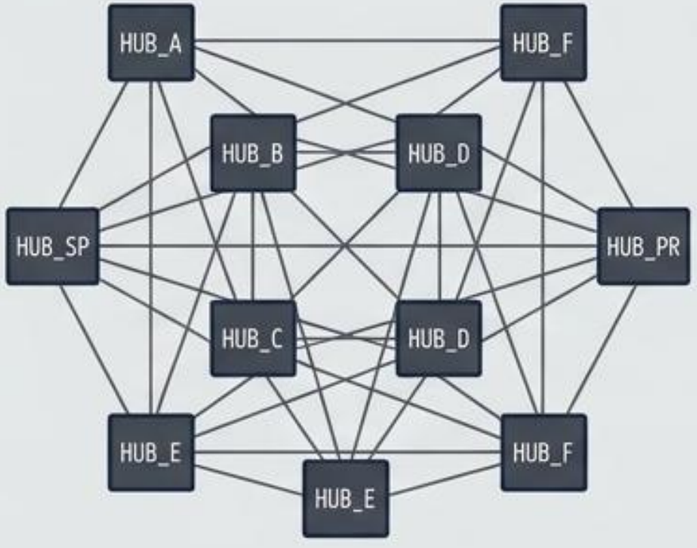
A magia do Cypher: O operador *1..3 substitui o que seriam três JOINs complexos e aninhados em SQL por uma simples instrução de profundidade de salto, executada linearmente pelo motor.



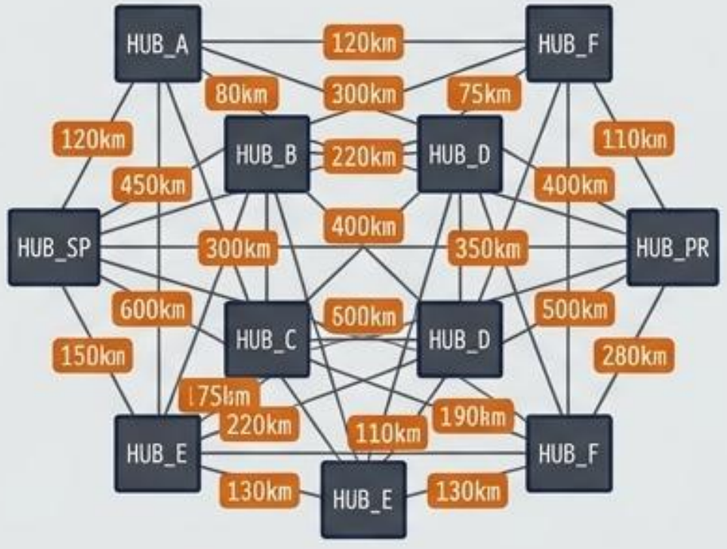
Query 2: Roteamento Inteligente (Algoritmo de Dijkstra)

```
MATCH path = shortestPath( (start:Hub {id: 'HUB_SP'})-[:CONNECTS_TO*]->(end:Hub {id: 'HUB_PR'})
)
RETURN path, reduce(totalDist=0, rel IN relationships(path) | totalDist + rel.distance_km)
as total_distance_km;
```

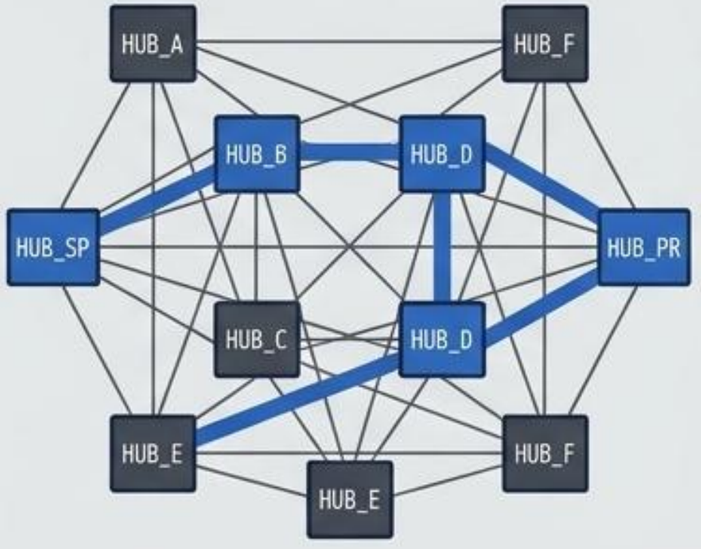
1. Mapeamento de Topologia

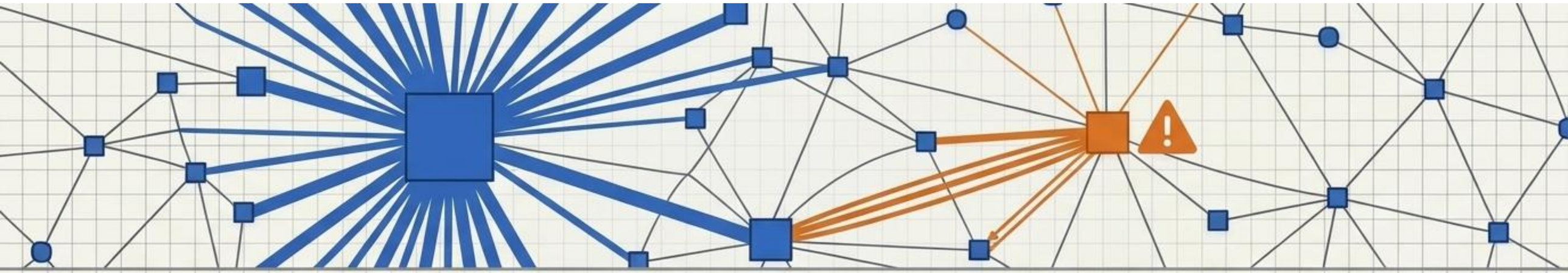


2. Avaliação de Pesos (Distância)



3. Resolução de Caminho Mínimo (Shortest Path)





Inteligência Operacional: Análise e Gargalos da Rede

Análise de Hubs

Objetivo: Encontrar Hubs com maior saída.

```
MATCH (hub:Hub)-[r:CONNECTS_TO]->()
RETURN hub.name, count(r) as
outgoing_connections
```

Detecção de Gargalos

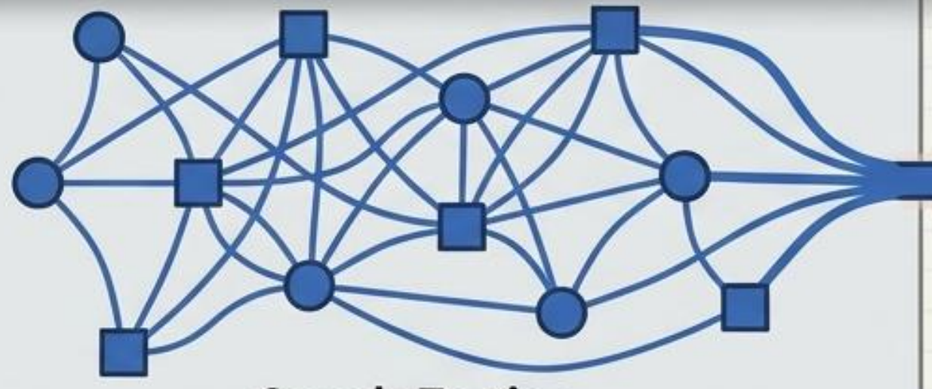
Objetivo: Motoristas com mais de 2 entregas ativas.

```
MATCH (driver:Driver)-[:DELIVERED {status:
'in_transit'}]->(customer)
WITH driver, count(customer) as
active_deliveries WHERE active_deliveries > 2
RETURN driver.name, active_deliveries
```

O Ápice Híbrido (Cypher + SQL JOIN)

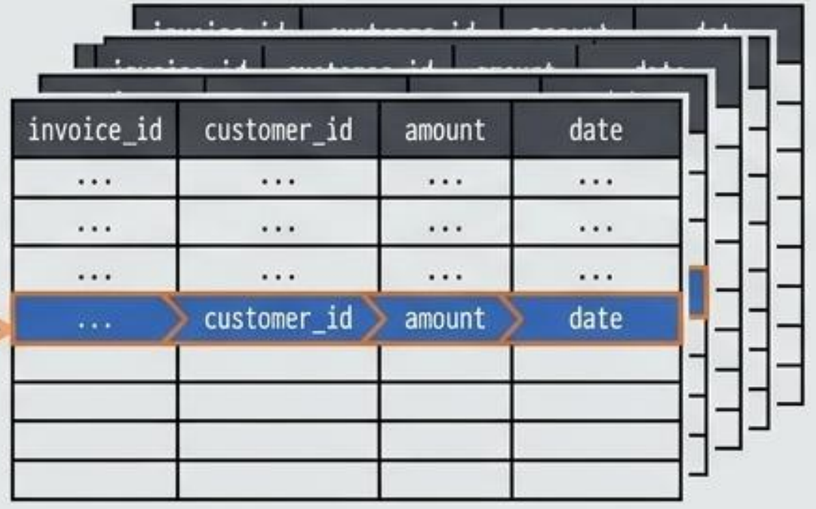
O verdadeiro poder do Apache AGE: Você não precisa escolher.

```
-- Executando Cypher para o traversal complexo
MATCH (driver:Driver)-[:DELIVERED]->(customer:Customer)
WHERE driver.id = 'DRV_001'
RETURN driver.name, customer.name, customer.type
-- [Conceito] O output gerado aqui pode sofrer JOIN direto
-- com a tabela SQL 'invoices' de faturamento!
```



Graph Engine

Data Pipeline



SQL Engine

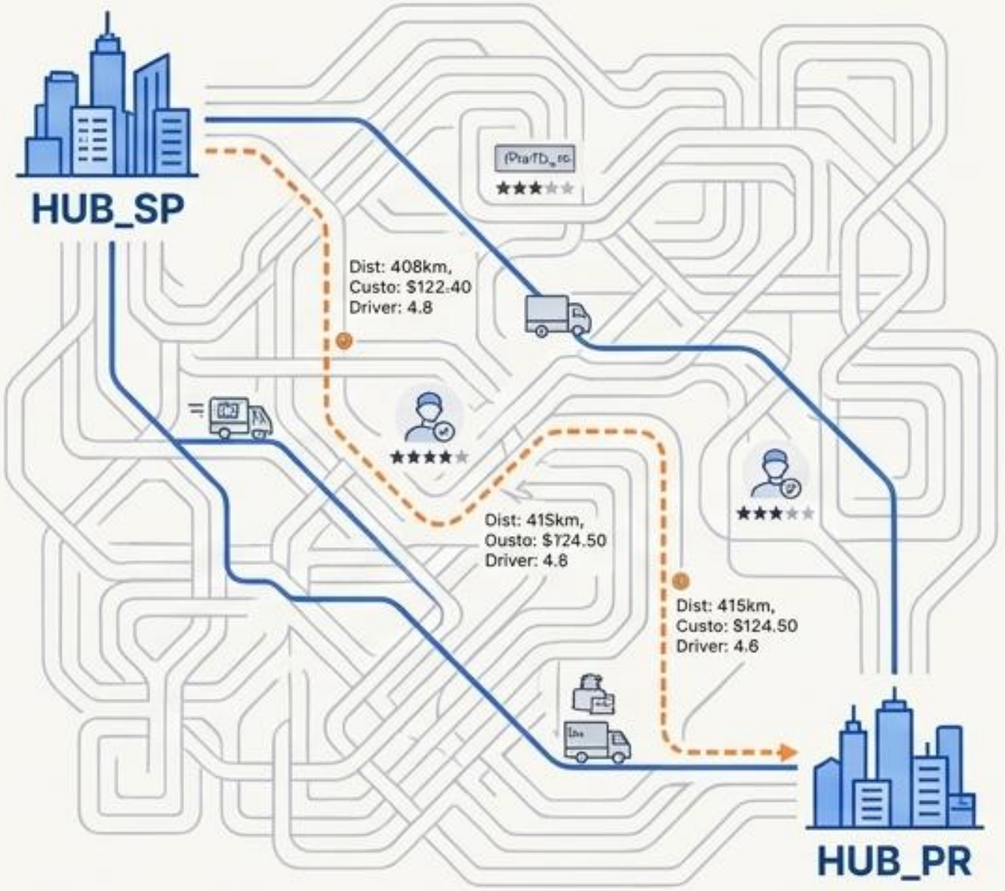
Operações analíticas complexas de rede (**Grafos**) alimentando processos transacionais financeiros (**SQL ACID**) em um único ciclo computacional.

Desafio Prático: A Missão de Roteamento

Criar uma query Cypher que identifique a **rota otimizada** de **custo mínimo** para entregar pacotes de **São Paulo** (**HUB_SP**) até **Curitiba** (**HUB_PR**).

Restrições Técnicas

- ✓ **Custo Total**: Calcular (`distância em km × custo_por_km`).
- ✓ **Viabilidade:** Filtrar estritamente por veículos com capacidade de carga suficiente.
- ✓ **Qualidade:** Priorizar motoristas com `rating > 4.5`.
- ✓ **Output:** Ordenar por custo crescente e retornar o `Top 3` rotas viáveis.



Dica: Explore os caminhos usando `allShortestPaths` ou defina a profundidade limite com `*1..3`.

Critérios de Entrega e Sucesso

O Pacote de Entrega



1. **Grafo** Populado: Mínimo de **10 hubs, 15 motoristas, 10 veículos, 20 clientes** + relacionamentos.



2. Arquivo **queries_logistica.cypher**: Contendo as **4 queries base** + a solução do desafio prático.



3. **Relatório Técnico (Markdown)**: Justificativa de uso do grafo vs SQL, comparativo de performance, e limites do AGE.

Pesos de Avaliação

30%	**Queries Cypher** (Sintaxe e Eficiência)
25%	**Modelagem do Grafo** (Nós/Arestas)
25%	**Desafio Prático** (Resolução de Custo)
20%	**Documentação Técnica** (Clareza)

Radar de Aprofundamento e Próximos Passos

Leitura Recomendada

Documentação Oficial: age.apache.org e opencypher.org

Livro O'Reilly: Graph Algorithms - Mark Needham & Amy Hodler.

Artigo Neo4j: When to Use Graph Databases.



Próxima Etapa

Prepare-se para a transição de volume.

Semana 4: Entrando na era do Big Data com Hadoop e ecossistemas distribuídos. Da modelagem complexa para a escala massiva.